

MapReduce Program using Python

1.Import the necessary modules.

- To conduct the reduction operation, we import the functools module reduce.

-To generate a dictionary with default word counts, we import the collections module

defaultdict.

2. The mapper function is described as follows:

Given a text as input, it separates it into words, and then produces a list of tuples, each of which includes a word and the value 1. The term is represented by the number 1, which will be used to count the occurrences later on in the reduction phase.

3. Establish the reducer function:

Word_counts and pair are the two inputs that the reducer function requires. pair is a tuple that contains a word and its occurrence count (from the word_counts dictionary, which records the current word counts).

4. The word count in the word_counts dictionary is updated by the reducer function.

When a term is not yet in the dictionary, it utilizes the defaultdict feature to manage the situation and sets its count to 0.

The function updates the word_counts dictionary and returns it after increasing the word's count in the dictionary depending on the value in the pair tuple.

5. List the texts you want to enter:

Three strings are defined in the list input_texts, which serves as the word counting input data. A sentence or document is represented by each string.

6. The map step involves applying the mapper function to each member of the input_texts list to produce a list of mapped_data.

Words and their definitions are represented by lists of tuples in each element of the mapped_data.

7. Flatten the mapped data:

A list of lists of tuples makes up the mapped data. We flatten the list of lists into a single list of tuples in this phase. A word and its count are contained in each tuple.

Reduce Phase: To combine the tuples depending on their words, we utilize the reduce() function together with the reducer function in the reduce phase. In order to guarantee that new words are automatically initialized with a count of zero, the defaultdict(int) is used as the initial value for the reduction process.

8. A word count display: The word counts for each word discovered in the input texts are contained in the `word_counts` dictionary, which is printed last.

The program's output gives the word counts for each distinct word that appears in the input texts.

Program 1:

```
from collections import defaultdict

# Step 1: Define your data
data = [
    ("apple", 1),
    ("banana", 2),
    ("apple", 3),
    ("banana", 4),
    ("cherry", 5),
]

# Step 2: Map function (key-value pair generation)
def mapper(item):
    word, value = item
    return [(word, value)]

# Step 3: Shuffle and Sort (group by key)
def shuffle_and_sort(mapped_items):
    shuffled = defaultdict(list)
    for key, value in mapped_items:
        shuffled[key].append(value)
    return shuffled.items()

# Step 4: Reduce function (aggregation)
def reducer(item):
    key, values = item
    return (key, sum(values))

# Step 5: Map Phase
mapped_data = [pair for item in data for pair in mapper(item)]

# Step 6: Shuffle and Sort Phase
shuffled_data = shuffle_and_sort(mapped_data)

# Step 7: Reduce Phase
reduced_data = [reducer(item) for item in shuffled_data]
```

```
# Print the final result
for key, value in reduced_data:
    print(f"{key}: {value}")
```

Program 2:

```
from functools import reduce
from collections import defaultdict
```

```
def mapper(text):
    # Split the text into individual words and return a list of (word, 1) tuples
    return [(word, 1) for word in text.split()]
```

```
def reducer(word_counts, pair):
    # Increment the count of the word in the word_counts dictionary
    word, count = pair
    word_counts[word] += count
    return word_counts
```

```
# Create a list of input texts (sentences or documents) to process.
```

```
input_texts = [
    "Hello world",
    "MapReduce is powerful",
    "Hello world and MapReduce"
]
```

```
# Step 1: Map - apply the mapper function to each input text
```

```
mapped_data = map(mapper, input_texts)
```

```
# Step 2: Flatten the mapped data into a single list of (word, 1) tuples
```

```
flattened_data = [pair for sublist in mapped_data for pair in sublist]
```

```
# Step 3: Reduce - apply the reducer function to the list of tuples to get the
word counts
```

```
word_counts = reduce(reducer, flattened_data, defaultdict(int))
```

```
# Step 4: Display the word counts
```

```
print(dict(word_counts))
```

Program for Spark:

Basic PySpark Setup

```
from pyspark.sql import SparkSession
```

```
# Initialize a SparkSession
```

```
spark = SparkSession.builder \
    .appName("MyPySparkApp") \
    .getOrCreate()
```

Creating and Viewing DataFrames

```
data = [("Mikhail", 15), ("Zaky", 13), ("Zoya", 8)]
```

```
df = spark.createDataFrame(data, schema=["name", "age"])
```

```
df.show()
```

From a CSV File: Use the DataFrameReader to load external files [Use your data file]

```
df = spark.read.csv("path/to/file.csv", header=True, inferSchema=True)
```

PREGEL Program in C++

```
#include <iostream>

#include <vector>

#include <limits.h>

using namespace std;

// Structure to represent an edge (neighbor + weight)
struct Edge {

int to ; // destination vertex

int weight; // weight of the edge

};

int main() {

int V, E;


// Take number of vertices from user
cout << "Enter number of vertices: ";

cin >> V;

// Take number of edges from user
cout << "Enter number of edges: ";

cin >> E;

// Create adjacency list for graph
vector<vector<Edge>> graph(V);

// Input edges
cout << "Enter edges (source destination weight):\n";

for (int i = 0; i < E; i++) {
```

```

int u, v, w;

cin >> u >> v >> w;

// Add edge from u to v with weight w
graph[u].push_back({v, w});

// If graph is undirected, uncomment below line
// graph[v].push_back({u, w});
}

int source;

// Take source vertex
cout << "Enter source vertex: ";
cin >> source;

// Distance array initialized with infinity (INT_MAX)
vector<int> dist(V, INT_MAX);

// Distance of source vertex is 0
dist[source] = 0;

bool changed = true; // To check if any update happens
int superstep = 0; // Counter for Pregel iterations

// Pregel-like iteration (supersteps)
while (changed) {
    changed = false; // Assume no changes initially
    superstep++;

    cout << "\n--- Superstep " << superstep << " ---\n";

    // Copy old distances (simulate message passing)
    vector<int> newDist = dist;

    // Loop through all vertices

```

```

for (int u = 0; u < V; u++) {
    // Skip unreachable vertices
    if (dist[u] == INT_MAX) continue;
    // Process all neighbors of vertex u
    for (auto edge : graph[u]) {
        int v = edge.to;
        int weight = edge.weight;
        // Relaxation step (like shortest path update)
        if (dist[u] + weight < newDist[v]) {
            newDist[v] = dist[u] + weight;
            changed = true; // Mark that update happened
            // Simulating message passing
            cout << "Vertex " << u
                << " sends message to " << v
                << " with distance " << newDist[v] << endl;
        }
    }
}

// Update distances after superstep
dist = newDist;

// Final output of shortest distances
cout << "\nFinal shortest distances from source " << source << ":\n";
for (int i = 0; i < V; i++) {
    if (dist[i] == INT_MAX)
        cout << "Vertex " << i << " : INF\n"; // Not reachable
}

```

```
else
cout << "Vertex " << i << " : " << dist[i] << endl;
}
return 0;
}
```